Assignment 1

Palaash Goel (2025AIZ8825) Adapters: Google Drive Link

All runs use seed 42 and the supplied files in their given order, with no resplitting or subsampling. Generation is greedy for evaluation, and every sampling parameter is set explicitly.

1 Part A: Where Do You Put the Adapters?

Experimental Setup. I train Qwen2.5-1.5B-Instruct on 20,000 training samples and then validate it on the 2,000 validation samples provided in the dataset.
Hyperparameters. In order to maintain fairness, I keep various hyperparameters constant across all runs unless there are some changes required specifically by the assignment. Each model is trained for 1 epoch using the AdamW optimizer and a learning rate of 2×10^{-4}. The learning rate warms up linearly over the initial 10% of training steps after which it decays linearly. A batch size of 4 is used with gradients accumulated over 8 steps (effective batch 32). The maximum sample length is 1024. LoRA is used with dropout (probability = 5%). Lastly, for QLoRA (configuration A6), the base model is loaded in 4-bit NF4 with double quantization and uses the same adapter setup as A4.

Run	target_modules	r	α	quant	trainable	%	mem (GB)	time (s)	tok/s	train loss	val loss	$F1_{IoU}$	$F1_{exact}$	leak
A1	q,v	8	16	none	1.09M	0.071	16.32	823	9563	0.0392	0.0197	0.854	0.810	1.75%
A2	q,k,v,o	8	16	none	2.18M	0.141	17.57	961	8181	0.0318	0.0153	0.881	0.837	1.63%
A3	attn + MLP	8	16	none	9.23M	0.595	22.34	1434	5484	0.0246	0.0107	0.913	0.870	1.24%
A4	attn + MLP	32	64	none	36.93M	2.336	22.82	1494	5264	0.0197	0.0092	0.926	0.882	**0.97%**
A5	attn + MLP	32	16	none	36.93M	2.336	22.82	1504	5229	0.0246	0.0108	0.918	0.874	1.09%
A6	attn + MLP	32	64	4bit	36.93M	2.336	27.60	2486	3165	0.0200	0.0096	**0.929**	**0.885**	1.01%

Table 1: Results of the sweep on the validation data. "attn + MLP" refers to q, k, v, o, gate, up, and down projections. While A6 performs the best in terms of $F1_{IoU}$ and $F1_{exact}$, A4 is able to meet much of A6's performance while also demonstrating a notably lower leak rate.

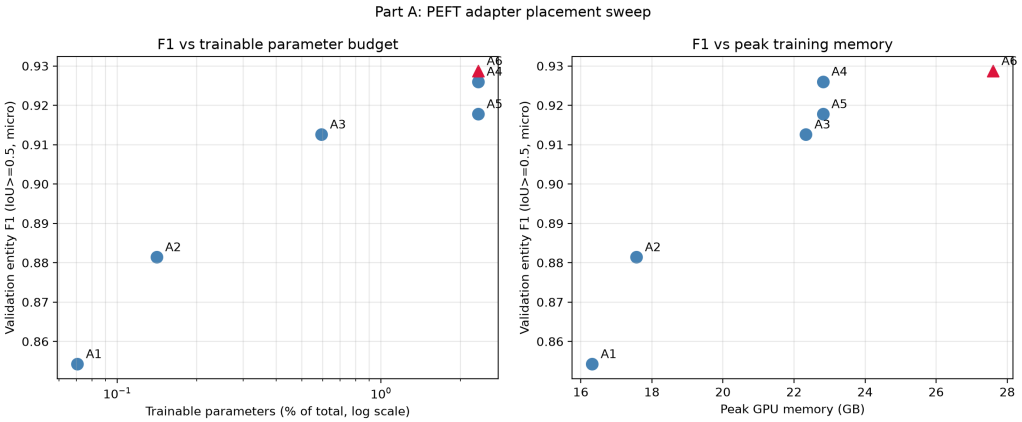


Figure 1: Validation F1 (IoU≥0.5, micro) vs. trainable % on a log scale (left) and vs. peak GPU memory (right).

Axis	ΔF1	F1 per +1M params	F1 per +1 GB	F1 per +100 s
Placement: q,v → q,k,v,o (A1→A2)	+0.027	**0.0249**	0.0218	0.0196
Placement: attention → attention + MLP (A2→A3)	+0.031	0.0044	0.0065	0.0066
Rank: r 8→32, α/r fixed at 2 (A3→A4)	+0.013	0.0005	**0.0281**	**0.0224**
α: 16→64 at r=32 (A5→A4)	+0.008	free	free	free

Table 2: Per-axis F1 gain to determine which factor contributes to the most F1 per unit of trainable parameters.

Analysis. The results from Table 1 demonstrate how the LoRA setup affects downstream performance, with A6 (QLoRA on attention and MLP) attaining the highest F1 scores while A4 (regular LoRA on attention and MLP) attains the lowest leak rate with competitive F1 scores. Variants A1 - A3 lag behind, perhaps due to their notably fewer trainable parameter counts. Additionally, while A5 has the same number of trainable parameters as A4 and A6, its lower α (scaling factor) leads to smaller gradient-based updates and slower convergence.
F1 per Unit of Trainable Parameters. As evident from Table 2, the *placement axis* contributes the most to

the F1 score for a fixed number of parameters. For instance, adding k,o to q,v gains 2.49 F1 points per 1 million additional parameters, as compared to a mere 0.05 points when rank is increased from 8 to 32. Note that since changing α does not change the number of parameters, none of its effects on F1 can be attributed to the number of trainable parameters in the model.

F1 per Unit of Memory (GB) and Wall-Clock Time (Second) The *rank axis* yields the highest increase in F1 with respect to both, increasing units of memory and wall-clock time. As evident, per GB of additional memory, increasing the rank from 8 to 32 yields a 2.81% higher F1. A similar trend can be seen for wall-clock time where it demonstrates a 2.24% increase in F1 per 100 seconds of additional run-time. Despite increasing the rank, the LoRA weights remain much smaller when compared to the entire activation memory, adding only 0.48 GB and 60 seconds when transitioning from A3 $\rightarrow$ A5. This allows it to eek out improvements for relatively small costs along these two axes.

Where the Curve Flattens. The curve flattens after A3. Four times more parameters (A3$\rightarrow$A4) yields 1.3 F1 points, compared with 5.8 points from A1$\rightarrow$A3.

The Cost of QLoRA. In a surprising turn of events, not only did QLoRA *not* cost any accuracy (it performed the best among all), it actually ended up costing *more* memory too by about 4.8 GB! I hypothesize that this is due to `prepare_model_for_kbit_training` upcasting the non-quantized layers (embeddings, norms, LM head) to fp32. 32-bit activations and logits over the 152K vocabulary can easily outweigh the $\approx$2 GB saved by 4-bit base weights, thereby leading to a net increase in memory consumption.

Best checkpoint: A4. While A6 has the highest F1, A4 is within 0.003 F1 of it and boasts of the lowest validation leak rate of all runs (0.97%). Additionally, it uses 17% less memory at its peak and trains 40% faster. A4 also runs on a full-precision base, so it can be merged for fast inference with no dequantization. I believe that for a sanitizer, the lower leak rate matters more than 0.3 points of F1 which is what motivated me to choose A4 as the best checkpoint instead of A6.

2 Part B: How Good Is Your Sanitizer, Really?

B.1 Span recovery. (1) Placeholders are found with the regex `\[LABEL(_N)?\]`, and any numeric suffix is dropped. (2) Placeholders are removed from the output, and the remaining literal text is aligned to `source_text` using `difflib.SequenceMatcher`. (3) A placeholder's source region runs from just after the last aligned character before it to the first aligned character after it. (4) Placeholders with no literal text between them split that region into equal parts. (5) Leading and trailing whitespace and punctuation are trimmed. (6) Adjacent spans with the same label, separated only by whitespace, are merged. Text deleted without a placeholder produces no span, but it does count toward over-masking. *Failure modes:* (i) the equal split in rule 4 misplaces boundaries between adjacent entities of unequal length, such as a given name and a surname; (ii) if the model rewrites nearby non-PII text, the unaligned region grows and the predicted span becomes too wide; (iii) rule 6 fuses two adjacent gold entities that share a label; (iv) short coincidental character matches inside a replaced value can split or shrink a region; (v) trimming removes punctuation that belongs to a gold span.

B.2 Metrics. Multiple metrics are used to evaluate sanitization performance. *Exact matching* is performed to check for identical offsets and labels. *Partial overlap* relaxes this by allowing matches where IoU$\geq$0.5 instead of only allowing perfect matches with respect to gold labels. *Macro F1* averages F1 scores over all classes, giving an aggregated measure of model performance. *Leak rate* is the fraction of sensitive values that appear in the output after lowercasing and whitespace collapsing. The match must not sit inside a longer alphanumeric token, so "12" does not match inside "123". *Over-masking* is the fraction of non-whitespace, non-PII source characters hit by a replace or delete operation when source and output are aligned. *Well-formed* means the output is non-empty and every bracketed tag is a known label. *Detection F1* is IoU matching with labels ignored.

B.3 Baselines. (1) *Regex*: a set of priority-ordered rules written to filter emails, URLs, geo-coordinates, IPv4/IPv6 addresses, credit card numbers, dates, time, SSNs, phone numbers, postcodes, @-based usernames, passwords (tokens with all four character classes), titles and sex. (2) *Zero-shot* and (3) *5-shot* both use Qwen2.5-1.5B-Instruct with no adapter and the same prompt plus the label list. The 5 exemplars are chosen deterministically from `partA_train` to cover the most labels. All model-based systems decode greedily, generating a maximum of 224 new tokens.

Comparison. As demonstrated by the results in Table 3, the fine-tuned sanitizer beats each baseline on every available metric and has a leak rate 21$\times$ lower than the best baseline (5-shot). Regex is precise (0.849) on formatted classes such as EMAIL, TIME and SSN (Table 4). However, it has no rule for names, usernames that do not start with "@" or ID, license and passport numbers, which scores 0 F1 on five of the top 10 classes and leaks 66.7% of values. The prompted baselines rewrite text (over-masking 13.7% and 48.8%) and often produce malformed output (5-shot is only 70.7% well-formed). Zero-shot goes a step further by leaving most PIIs untouched. The main remaining weakness of A4 is telling similar ID types and name parts apart (PASSPORT P=0.842, LASTNAME1 P=0.838). Lastly, exact F1 is 3.9 points below IoU F1, which indicates that there are some boundary issues in A4 which end up racking up penalties on the former metric but are acceptable under the latter.

System	Exact span + label				IoU≥0.5 + label				Detect.		
	P	R	F1	macro F1	P	R	F1	macro F1	F1	Leak ↓	Over-mask ↓ / WF
Fine-tuned A4	**0.890**	**0.904**	**0.897**	**0.848**	**0.929**	**0.944**	**0.936**	**0.920**	**0.969**	**0.79%**	0.57% / 99.4%
Regex	0.742	0.267	0.392	0.241	0.849	0.305	0.448	0.312	0.486	66.73%	0.68% / 97.5%
5-shot Qwen	0.484	0.429	0.454	0.436	0.524	0.464	0.492	0.483	0.696	16.86%	13.69% / 70.7%
Zero-shot Qwen	0.270	0.070	0.111	0.102	0.288	0.075	0.119	0.110	0.226	53.96%	48.84% / 81.1%
A4 on OOD (raw labels)	0.208	0.147	0.173	0.117	0.231	0.164	0.192	0.134	0.749	30.14%	0.18% / 95.9%
A4 on OOD (labels mapped)	0.537	0.380	0.445	0.276	0.585	0.414	0.485	0.320	0.749	30.14%	0.18% / 95.9%

Table 3: Performance metrics on the `partB_test` dataset (n=2000 samples). I also evlauate A4 on the out-of-distribution `partB_ood_test` dataset (n=1000 samples) to test its generalizability. WF = well-formed rate.

Class	Support	Fine-tuned A4 (IoU)			Baseline F1 (IoU)		
		P	R	F1	Regex	5-shot	0-shot
TIME	945	0.925	0.957	0.941	0.787	0.683	0.146
USERNAME	760	0.957	0.947	0.952	0.000	0.498	0.129
SOCIALNUMBER	668	0.954	0.960	0.957	0.694	0.671	0.176
EMAIL	662	0.972	0.985	0.978	0.966	0.788	0.262
IDCARD	651	0.929	0.923	0.926	0.000	0.527	0.137
DRIVERLICENSE	643	0.949	0.890	0.918	0.000	0.431	0.134
LASTNAME1	623	0.838	0.891	0.864	0.000	0.292	0.035
BOD	553	0.969	0.975	0.972	0.705	0.443	0.011
PASSPORT	552	0.842	0.949	0.893	0.000	0.472	0.201
SEX	529	0.963	0.987	0.975	0.564	0.523	0.018

Table 4: Per-class results for the 10 most frequent gold classes in `partB_test`.

Error taxonomy. Of 12,779 gold entities, 721 are unmatched under IoU≥0.5 + label, and 918 of 12,976 predictions are unmatched.

In total, 101 gold values present in 70 out of 2,000 examples leak verbatim. Most of the leaked classes are low-support or ambiguous classes such as LASTNAME1, TIME, DATE, USERNAME and COUNTRY. A wrong label does not leak anything, since the value is still masked. Only 84 of the 721 misses (11.7%) output the sensitive value verbatim without any masking whatsoever. A detailed error taxonomoy is present in Table 5.

Generalization (OOD). On `partB_ood_test`, IoU F1 falls from 0.936 to 0.192, while label-agnostic detection F1 stays at 0.749 (P 0.903, R 0.639). While it might seem that my sanitization model suffers from poor generalizability, there are two logical reasons for this decline in performance that suggest otherwise. *(1) Label vocabulary shift.* The OOD file uses a notably different set of labels (FIRSTNAME, PREFIX, PASSWORD, ZIPCODE, IPV6, ...) than the ones that the model was trained to predict (GIVENNAME1 instead of FIRSTNAME, PASS instead PASSWORD, POSTCODE instead of ZIPCODE, IP instead of IPV4/V6). As a result, out of 2,179 missed OOD entities, 1,546 correspond to cases where the model identifies the sensitive span correctly but simply assigns a different label name than what the OOD dataset expects (e.g. FIRSTNAME→GIVENNAME1 (128), PASSWORD→PASS (69), LASTNAME→LASTNAME1 (69)). For a more comprehensive evaluation, I manually map each OOD class label of an identified span to a label from the training vocabulary. Doing so raises IoU F1 to 0.485, indicating that the sanitizer performs more robustly than indicated earlier. *(2) Unseen classes.* Despite the mapping earlier, there are 980 gold values (30.1%, in 636 of 1,000 examples) belonging to classes that do not have a counterpart in the training dataset and thus, could neither be mapped nor be identified by the sanitizer. Examples of such classes include JOBAREA (69), ACCOUNTNAME (63), URL (59), JOBTITLE (49) and AMOUNT (45). Despite this, A4 has a relatively high OOD detection precision of 0.903, indicating its ability to detect sensitive spans while simultaneously being unable to assign OOD-aligned labels to these spans.

3 Part C: Training Data Extraction Attacks

C.1 Victims. V-raw and V-san are Qwen2.5-0.5B with identical LoRA settings: q,k,v,o projections, r=32, α=64 and dropout of 0.05. The other training hyperparameters are kept exactly the same as those in Section 1. `partC_corpus_sanitized.jsonl` was built by sanitizing all 8,007 raw rows using the A4 sanitizer. Canaries were not handled separately so that sanitizer performance could be judged more clearly. Comparing the two corpora afterwards, the sanitizer masked the secret in all 425 canary rows (tagging it `[PASSPORT]`), masked 198 of the 200 targeted member values, and left 18 of the 214 tracked PII values somewhere in the corpus. Text length was mostly preserved (mean character ratio 0.97, with only 13 of 5,020 documents losing more than 40% of their characters). 448 documents came back byte-identical, though not all of them contain PII. Held-out perplexity is token-weighted over all 500 held-out documents.

C.2 Attacks. *(a) Discoverable:* for each distinct document with ≥100 tokens, the first 50 tokens are the prompt and we check whether greedy decoding reproduces the next 50 tokens exactly. *(b) Untargeted:* 5,000 samples are drawn from the start token (top-k 40, temperature 1, 128 tokens) and ranked via membership scoring. A sample

Miss type (gold entity unmatched)	Count	Common Examples (Support)
Right span, wrong label (IoU≥0.5)	416 (57.7%)	DRIVERLICENSE→PASSPORT (45), GIVENNAME2→LASTNAME1 (27), IDCARD→PASSPORT (23), GIVENNAME1→LASTNAME1 (20)
Missed, value left verbatim	84 (11.7%)	LASTNAME1 (16), TIME (10), DATE (8), USERNAME (8), TITLE (6)
Overlap with wrong label and IoU<0.5	84 (11.7%)	–
Right label, bad boundary (IoU<0.5)	76 (10.5%)	span-recovery failure modes (i)–(iii)
Missed, value not verbatim (text altered)	61 (8.5%)	model rewrote or dropped the region with no placeholder
Spurious prediction (no gold overlap)	322	TIME (54), GIVENNAME1 (33), DATE (25), IDCARD (19)
Malformed output / output <50% of source length	12 / 1	out of 2,000 examples

Table 5: A taxonomy for the errors demonstrated by the model.

counts as a member if it shares a verbatim 16-token window with a training document. PII recovery counts how many of 214 values (member gold values from `partC_targeted` plus the 20 canary secrets) appear in the samples. *(c) Canary:* each true secret's template sentence is ranked by total NLL against 9,999 random 6-digit alternatives, and exposure $= \log_2 10^6 - \log_2(\text{rank}/10^4 \cdot 10^6)$. *(d) Targeted:* each prompt is decoded greedily and with 10 samples (top-k 40, 32 tokens) and checked if a gold value was leaked or not.

	Metric	V-base	V-raw	V-san
Utility	Held-out PPL (token-weighted) ↓	13.63	**8.61**	11.18
	Training tokens per epoch	–	977,228	761,665
(a) Discoverable	Exact match of 50-token suffix (3,520 docs)	0.0%	0.0%	0.0%
	by n_{reps} = 1 / 2 / 8 / 32	0 / 0 / 0 / 0%	0 / 0 / 0 / 0%	0 / 0 / 0 / 0%
	Mean token edit similarity	0.171	0.209	0.185
(b) Untargeted	Unique samples of 5,000	5,000	4,994	4,998
	P@100: ppl / ref / lower / zlib	0 / 0 / 0 / 0	.09 / .00 / .04 / .16	0 / 0 / 0 / 0
	Distinct training docs matched	3	84	17
	PII values recovered (of 214)	16 (7.5%)	35 (16.4%)	18 (8.4%)
(c) Canary	Mean exposure, bits (20 canaries)	2.00	7.50	1.13
	by n_{reps} = 1 / 4 / 16 / 64	1.40 / 2.10 / 1.38 / 3.14	1.77 / 5.57 / 9.37 / 13.29	0.94 / 0.47 / 1.96 / 1.13
(d) Targeted	Member: greedy / any-of-10 sampled	0.5% / 0.5%	0.0% / 1.5%	0.0% / 1.0%
	Control: greedy / any-of-10 sampled	1.0% / 2.0%	0.5% / 3.0%	0.5% / 1.5%

Table 6: Base / V-raw / V-san comparison. All three models are attacked with the same settings against the raw corpus. V-base has no adapter, so its reference model is itself and $s_{\text{ref}} \equiv 1$. n_{reps} refers to the number of repetitions involved in the test.

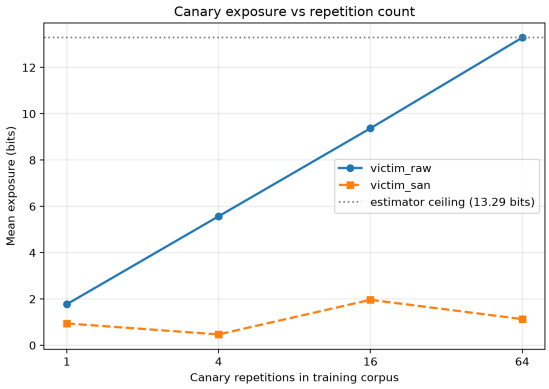


Figure 2: Mean canary exposure vs. repetitions (5 canaries per bucket).

Exposure. As evident from Figure 2, on V-raw, exposure grows roughly linearly in $\log(n_{\text{reps}})$, from 1.8 bits at 1 repetition to 13.3 bits at 64. The untouched base model sets a low noise floor of 1.4–3.1 bits (mean 2.00) since it has never seen the secrets. V-raw's single-repetition canaries (1.77 bits) fall inside that noisy range, so memorization first becomes detectable at 4 repetitions. V-san stays at 0.5–2.0 bits with no trend in repetitions. That is within the noise range, as expected, since every canary secret was masked before training. *Estimator limitation:* the sampled rank is at least 1, so the estimate cannot exceed $\log_2(10^4) = 13.29$ bits, whereas the true maximum over $|R|{=}10^6$ is 19.9 bits. All five 64-repetition canaries on V-raw sit exactly at this ceiling (rank 1 of 10,000), so their true exposure is unknown and the 64-rep point is a lower bound.

Discoverable and targeted. After 1 epoch, no model reproduces a 50-token suffix exactly, even for documents repeated 32 times. Edit similarity does rank the models as expected: V-raw 0.209 > V-san 0.185 > base 0.171. V-san sits above base because it trained on the same non-PII text. Targeted leak rates are at noise level for all three models. Controls leak as often as or more often than members, and the untrained base model already leaks 1/200 member values greedily and 4/200 controls under sampling. These hits are values guessable from the prompt context, so this attack gives no membership signal at this training budget.

C.3 Privacy–utility trade-off and traced leaks

Efficacy vs. cost. Leakage is best measured as the excess over the untrained base model, because the base model already "recovers" generic values. From that perspective, sanitization removed almost all of the leakage the attacks can detect. Mean canary exposure fell from 7.50 to 1.13 bits (base 2.00), and from 13.29 to 1.13 at 64 repetitions. PII values recovered by untargeted sampling fell from 35 to 18, against 16 for base: the excess over base went from 19 to 2 (an 89% reduction). Training documents matched by samples fell from 84 to 17 (base 3), and V-san's P@100 is 0 under every score, the same as base. Unfortunately, the higher privacy comes at a notable utility cost in the form of a held-out PPL of 11.18 instead of 8.61 (+30%). Importantly, a part of this gap is not a pure privacy cost. Specifically, V-san saw 22% fewer training tokens, and after sanitization, 1,311 more distinct documents are shorter than 100 tokens (2,811 vs. 1,500). Characters dropped by only 3%, so the likely cause is that placeholders take fewer tokens than the digit-heavy values they replace, not truncated documents. The held-out documents also contain raw PII formats that V-san never saw. The 18 values recovered from V-san are almost entirely guessable, since the untrained base model recovers 16. Separately, 18 of the 214 values survive somewhere in the sanitized corpus, and 13 of those are short generic strings that also appear in the never-trained held-out set (single-letter sex codes, 2-letter country codes, titles, short times).

Traced leaks. Only two of the 200 targeted member values reached V-san's training data unmasked. V-san's member leak rate (2/200 sampled) is at the base-model noise level, so the attacks cannot show that these particular values were memorized. They are, however, still real sanitizer failures that remain in V-san's training data. Both are "missed, value left verbatim" cases from the Part B taxonomy. *(1)* `member_004`*, DATE, document repeated 2×.* The value (20**-**-**) sits in free text ("follow-up appointments scheduled for ⟨V⟩ to review progress"). The sanitizer returned this 475-character document completely unchanged, indicating a pass-through failure rather than a single missed span. *(2)* `member_092`*, TITLE, document repeated 8×.* The value (Ca**) starts a list item, "1. ⟨V⟩ – `[EMAIL]` – `[SOCIALNUMBER]`". Interestingly, while the sanitizer was able to mask every other field on the line, it missed masking TITLE, which is among its most frequently leaked classes (Table 5).

4 Next Steps

Seeing the leaks, I would employ the following defensive strategies to build a more robust system: (1) *Deduplicate before training*, since repetition is what drives exposure from 1.8 to ≥13.3 bits. (2) *Pair the model sanitizer with a high-recall rule pass* for structured secrets (digit codes, keys, IDs) and add a check that no source token with high PPL survives unmasked. This aims to resovle the 84 verbatim misses and the OOD classes such as URL and AMOUNT, where the model under-masks. (3) *Replace sensitive text with pseudonyms rather than placeholders*, to allow V-san to keep realistic text statistics and lose less utility due to the loss of information and context propagated by placeholders.

5 Limitations and Honour Code

All results use a single seed and 1 training epoch for both parts; multi-seed and multi-epoch runs might have been more informative. Additionally, the OOD label mapping is crafted manually; perhaps an automated alternative could be figured out and compared with. QLoRA's memory blow-up was not investigated much beyond the library's apparent behavior.

Honour code. "Even though I have taken help from the following students and LLMs in terms of discussing ideas and coding practices, all my code is written by me." Students consulted: Aravind Shaj and Raj Vardhan Singh Tomar. LLMs/tools used: Claude.